

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.DOI

Deep Transfer Learning for Lip Reading Based on NASNetMobile Pretrained Model in Wild Dataset

ASHWAQ WALEED ABDUL AMEER¹, PEDRAM SALEHPOUR¹, and MOHAMMAD ASADPOUR¹

¹Department of Computer Engineering, Faculty of Electrical and Computer Engineering, University of Tabriz, Iran.

Corresponding author: Pedram Salehpour (e-mail: Psalehpour@tabrizu.ac.ir).

ABSTRACT Lip reading is extensively used in visual speech recognition to classify and recognize words by analyzing mouth movements in silent videos. Transfer learning models are often employed to improve performance in this area. However, the effectiveness of these models can be significantly affected by the quality of the data, especially when dealing with unstructured wild data. This includes scenarios where subjects are not directly facing the camera, words are not clearly articulated, or image cropping techniques are not used. In this study, we present a novel framework for visual speech recognition utilizing the NASNetMobile architecture. We evaluate the performance of this model on the GLips 2022 wild dataset, which is known for its unstructured and challenging nature. This lightweight model, with its low parameter count, is particularly well-suited for this task. The proposed framework involves a process that extracts features from video frames in a time sequence, employing methods such as Convolutional Neural Networks (CNN), CNN-Gated Recurrent Units (CNN-GRU), Temporal CNN, and Temporal PoinWise. We empirically demonstrate that NASNetMobile outperforms VGG19, DenseNet121, and MobileNetV3Large, which are used in existing models. The performance of NASNetMobile is evaluated using several key metrics. The model contains 4,680,739 parameters. It achieves an accuracy of 0.4840, an F1 score of 0.4850, and a weighted F1 score of 0.4850. Our findings show that NASNetMobile is more effective, increasing accuracy and F1 scores while reducing model complexity.

INDEX TERMS Lip Reading, Visual Speech Recognition, NASNetMobile, Transfer Learning.

I. INTRODUCTION

LIP Reading (LR) systems comprehend and categorize words by analyzing the movements of a speaker's lips, making this application particularly valuable when speech sounds are unavailable or unclear [7]. Interest in applying Deep Learning(DL) to LR systems has surged in recent years due to their diverse applications [14, 23] in various languages [14, 23]. These include authentication, human-computer interaction devices, hearing aids, voice assistants, surveillance, pronunciation correction, and health-care applications like synthesizing voice for non-verbal patients [28]. Converting video to text without voice, known as Visual Speech Recognition (VSR), is a captivating research field.

VSR faces challenges such as homophones (e.g., </p/, and/b/>, </k/, and/g/>), which have distinct sounds but similar mouth shapes. Additionally,

variations in the speaker's head position complicate VSR, while changes in lighting and spatial/temporal resolutions impact VSR application performance. Prior research has explored VSR models trained on both structured and wild datasets. While DL has been applied to structured datasets, yielding satisfactory results albeit with high complexity and parameters [37, 24], other studies have focused on wild datasets using less effective basic custom models [30, 26]. However, we opt for a different approach by utilizing pretrained models on authentic wild datasets with fewer parameters for improved performance.

Recent advancements in DL architecture models significantly enhance VSR performance on structured datasets using a variety of techniques. These models predominantly employ 2D or 3D Convolutional Neural Networks (CNN)-based [43, 45, 15] architectures to extract features from video frames, followed by

processing with LSTM [46], GRU [32, 8] or transformer [22, 27] models to capture temporal patterns from sequence of video frames, with or without attention mechanisms. However, the complexity of these models leads to heavyweight deep learning models, prompting the pursuit of lightweight alternatives to achieve optimal performance on structured datasets.

Transfer Learning (TL) is an approach used to address limited data availability by training on source datasets, such as the ImageNet dataset with its 1000 classes, using pretrained models [42]. Pretrained models can be employed in two ways: transfer learning and fine-tuning. In transfer learning, the pretrained model's parameters remain fixed, while in fine-tuning, these parameters are adjusted using target data after unfreezing the pretrained model.

Researchers, including ourselves, explore common pretrained models to tackle the VSR problem, employing them for both transfer learning and fine-tuning by different TL models such as ResNet [22, 13], DenseNet121 [40] and VGGNet [35]. We use NASNetMobile [38] as a pretrain model for TL in VSR research which has low parameters and needs to low memory and a few computational resources. Our research stands out from previous studies by utilizing pretrained models on wild datasets. While previous studies either relied on basic custom models with introduced wild datasets or used pretrained models on the structured dataset, we take advantage of pretrained models on wild datasets.

In the existing literature, two significant gaps have been identified. The first gap pertains to the use of heavy models that are impractical for mobile applications due to their high computational demands. Which leads to lower complexity enhances computational efficiency, speeds up inference times, and reduces the resource requirements. The second gap concerns the reliance on synthetic data for model testing, which does not accurately reflect real-world scenarios. This study addresses these gaps by introducing a lightweight TL model, NASNetMobile, and evaluating it on a real-world dataset. The key contributions of this paper are:

- Development of a lightweight model using NASNetMobile for feature extraction and CNN for temporal feature extraction, as illustrated in the proposed flowchart.
- Comprehensive analysis and evaluation of the proposed model using the GLips wild dataset, with preprocessing and augmentation techniques tailored for real-world data.
- Comparative performance analysis of NASNetMobile against VGG19, DenseNet, and MobileNetV3Large, using CNN, CNN-GRU, Temporal CNN, and PointWise convolution architectures.

II. RELATED WORK

In silent conditions, the VSR systems rely on computer vision techniques to convert video to text. In the Video-based application of VSR, identifying lip appearance, shape, or viseme identification is crucial, and various visual feature extraction techniques have been introduced to address this need. Currently, in VSR, the focus for feature extraction lies on the mouth or lip area, with other facial components like eyes, nose, and eyebrows being ignored. Feature extraction methods in VSR can be divided into two main categories: traditional manual methods and deep learning methods [28].

While some attempts at VSR have been made without the use of Machine Learning (ML) and DL techniques, they have yielded unsatisfactory results for wild datasets. Traditional manual feature extraction methods typically involve three main steps: extracting the region of interest (e.g., lip region or mouth) [13, 15], designing a feature extraction algorithm, and classification. Input images are typically in Gray-Scale or RGB format, and various video-based features are extracted, including Discrete Cosine Transform (DCT) [33], Linear Discriminant Analysis (LDA) [29], Histogram of Oriented Gradients (HOG), and Local Binary Patterns (LBP) [33]. Methods such as auto-encoders [46] and feature transformation techniques like feature space Maximum-Likelihood-Linear-Regression (fMLLR) [39] are also employed in the ML domain. Following feature extraction, classifiers such as Hidden Markov Models (HMM) [25], Support Vector Machines (SVM), k-nearest Neighbors (KNN), and Deep Neural Network k-Nearest Neighbors (DNN-KNN) [33] are utilized to classify phonemes, visemes, alphabets, and words in datasets.

Deep Learning (DL) outperforms traditional feature extraction methods by obtaining more relevant features autonomously. DL models offer the advantage of automating feature extraction and classification processes, as well as simplifying preprocessing tasks. VSR involves two main phases: front-end and back-end. Front-end algorithms employ various methods for feature extraction, including CNN, autoencoder-based Deep Bottle Neck Features (DBNFs) followed by Linear Discriminant Analysis (LDA) and fMLLR feature transformation [29], thirteen-layer CNNs, 2D-CNN/3D-CNN architectures, convolutional autoencoders [25], and truncated 3D-CNN Seq2seq transformer networks with time-masked self-attention [21]. The temporal relationship of the features extracted from each frame obtained from the first phase is taught. In the back-end phase, frames within video images are analyzed using LSTM, Bi-LSTM, GRU, Transformer-based seq2seq models, or Multi-Scale Temporal Convolutional

Table 1: Comparative analysis of samples of existing works.

References	DL model categorization	Face detection utilization	Dataset type
[12]	DL + TL (VGG16)	RGB-cropped mouth frames	Structured dataset
[2]	DL + TL (VGG19 AlexNet)	Lower face image	Structured dataset
[30]	DL model from scratch	Without face detection	Wild dataset
Ours	DL + TL (NASNetMobile, VGGnet, MobileNetV3, DenseNet)	Without face detection	Wild dataset

tional Networks (MSTCN) [44].

Significant progress has been achieved in front-end techniques due to the adoption of TL in recent years. Transfer learning has also demonstrated its effectiveness in various fields beyond lip reading. For instance, Liao et al [20] introduced a Kernel-based Channel Attention (KCA) and Mutual Information Domain Adaptation (MIDA) approach to enhance transfer learning. Their method addresses unsupervised domain adaptation (UDA), improving feature extraction and aligning feature distributions between source and target domains. This approach mitigates domain shift, a common challenge in transfer learning, especially when transitioning from synthetic to real-world data. Liao et al [19] further proposed a calibration-based dual prototypical contrastive learning (CDPCL) approach to address domain discrepancies in semantic segmentation within deep transfer learning. Their method combines uncertainty-guided PCL (UPCL) and hard-weighted PCL (HPCL) to learn domain-invariant class-wise features, significantly improving performance while maintaining computational efficiency. These studies highlight how transfer learning in other domains has achieved notable success, offering promising insights for its application in lip reading tasks. Various pretrained models such as VGGNet, ResNet34, ResNet18, AlexNet, DenseNet121, and ShuffleNet have been extensively utilized. Following this, back-end methods are applied, and various classification techniques are used to categorize words, visemes, speakers, sentences, and digits.

Datasets in this VSR field typically fit into two categories. The first consists of datasets tailored for LR tasks, where the face of the speaker is in front of the camera and lip movements are clear such as CMU AVPFV [16], SpeakingFaces [1] and DVS-lip [37]. Participants in these datasets are often aware of the task's objectives, leading to a bias towards accurately spoken words. The second category LSVSR [34] and MV-LRS [5], include data from real-world sources like films and TV shows, BBC and YouTube posing numerous challenges overlooked by previous studies. Recent research emphasizes the need to evaluate deep learning models on genuine datasets, despite their high success rates with synthetic data. Studies focusing less on real datasets have typically achieved ac-

curacies of 20 to 30 percent. This paper tackles challenges inherent in real-world datasets and employs transfer learning to outperform previous methods.

In wild datasets, domain diversity issues such as limited variations in language, speaker characteristics, and accents can bias deep learning model training [31]. Benster et al. [3] enhanced VSR by introducing Multimodal Orofacial Neural Audio (MONA) and Large Language Model Integrated Scoring Adjustment (LISA). Dong et al. [6] developed a wireless, unobtrusive silent speech interface to track and decode movements of the temporomandibular joint, addressing challenges like bulkiness and limited portability. Laux et al. [17] proposed a framework to improve communication for speech-impaired patients in intensive care through lip-reading, using frames of the patient's face to infer audio features as intermediate targets for predicting uttered text. Wang et al. [41] investigated silent speech recognition using depth sensing at on-wrist, on-head, and in-environment sensor locations. This research highlights the potential of depth sensing for VSR, showing how speech can be captured and interpreted without traditional audio input devices.

Numerous studies have delved into LR, using both audio and visual data. However, most investigations on VSL have focused on structured datasets. Fewer studies have explored wild datasets, with some researchers opting to crop and analyze lip images. This approach is illustrated in Table 1, As indicated in [28], eliminating image cropping is crucial for accurate LR in real-world scenarios. However, few studies have explored LR using the entire image, which is the primary focus of our research in this paper.

In Table 1, we find articles closely aligned with our research, focusing on DL and TL for silent LR in word classification type. These articles are categorized based on their approach to data collection and facial element processing. The field typically distinguishes between structured datasets, where individuals face the camera under controlled conditions, and real datasets sourced from internet videos and annotated by experts. While many studies focus on word classification in LR and some concentrate on speaker identification [35], our goal, along with the articles in Table 1, is to classify words independently of speaker identity. In the category of face detection

utilization, papers fall into two classes: those used face detection and those without. Our study, exemplified by articles 1 and 2, falls into the latter category, as we do not rely on face detection for LR. Instead, our focus is on overcoming image processing challenges in LR without resorting to cropping or isolating lip movements.

III. MATERIALS AND METHODS

The materials and methods section of our study is divided into four main categories: Dataset, preprocessing techniques, feature extraction models (which includes a detailed discussion of the NASNetMobile model and a brief overview of other transfer learning models), and the frame-based temporal relationship extractor.

A. DATASET DESCRIPTION

For our VSR project, we've chosen the GLips 2022 dataset [30], renowned for its broad and varied collection of video data. Classified as a wild dataset, it accurately represents real-life scenarios. With 500 classes, each containing spoken words of different lengths (ranging from 4 to 18 letters), and contributions from around 100 speakers, the dataset totals 250,000 videos. The videos in the GLips 2022 dataset maintain a standardized resolution of 256×266 , with a duration of 1.16 seconds and a frame rate of 25 frames per second (fps), allowing for effective analysis of lip movements and speech patterns. Sourced primarily from YouTube, these videos predominantly feature the German language, reflecting authentic speech patterns and linguistic nuances observed in online content during the year 2022.

The GLips dataset's presents challenges for VSR models, potentially limiting their ability to generalize across languages and conditions. Not implementing a speaker-independent split maintains a larger word set however affect generalization across speakers. Creating disjoint test sets through face recognition adds technical complexity, enhancing dataset diversity but complicating data management. Excluding data augmentation due to performance concerns may restrict the model's robustness. Developing GLips involves intricate processes like audio alignment and face detection, ensuring data quality and compatibility for machine learning tasks.

B. PREPROCESSING

Preprocessing is crucial for training DL models effectively, ensuring accuracy and robustness in real-world scenarios. We prioritize normalization and augmentation techniques as they significantly enhance data quality and model performance.

1) Normalization

Normalization is crucial for machine learning, as it scales dataset features to similar ranges, enhancing algorithm performance. A common method is Min-Max Scaling, which adjusts features to a range between 0 and 1 according to the following equation:

$$X_{\text{normalize}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

Here, X denotes the original value of each pixel in an image of the dataset, $X_{\min}$ represents the minimum value, $X_{\max}$ indicates the maximum value, and $X_{\text{normalize}}$ is the normalized value of X .

2) Augmentation

Data augmentation expands dataset size by applying various transformations to existing data samples, which is especially beneficial for limited datasets, enhancing diversity and variability to improve DL model generalization and robustness. Common techniques include image flipping, adding noise, and interpolation, the latter of which estimates new data points within existing data ranges.

Noise was introduced to the training data by adding normal noise with a zero mean and a variance of 0.23. Any resultant data points outside the range $[0,1]$ were clipped to fit within this range. Additionally, interpolation techniques were used to create new training data. For two training data points x_1 and x_2 from the same class, a new data point was generated by averaging them, calculated as $(x_1 + x_2)/2$.

C. FEATURE EXTRACTION MODEL

DL-powered VSR applications typically consist of two classification stages: front-end and backend. The front-end, known as the feature extraction model, processes video frames to extract features using DL models. The backend includes a step called the temporal relationship analyzer, which interprets the temporal relationships between frames using the features extracted by the front-end model.

The selection of TL models in this study is based on two main criteria: the number of parameters and the variance in the features extracted by the TL models. Our objective is to develop a lightweight model suitable for implementation on mobile devices. Previous research highlights the challenges posed by heavy models in lip-reading applications, underscoring the importance of considering the number of parameters. Furthermore, the variance of the extracted features is a significant scientific metric. Models with low feature variance lack informative value and are therefore excluded from selection. Consequently, we have chosen TL models with fewer than 20 million parameters and a feature variance exceeding 0.3. The selected TL models for feature extraction are NASNetMobile, VGG19, DenseNet121,

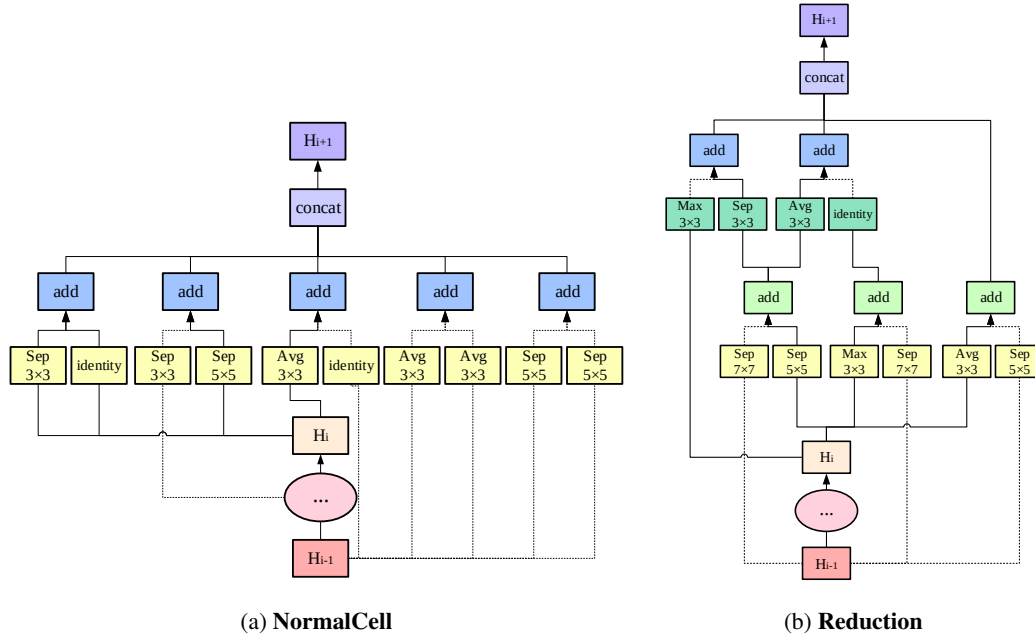


Figure 1: The design of the most optimal convolutional cells in NASNetMobile.

and MobileNetV3Large. Table 2 provides details on the number of parameters and feature extraction variances of these TL models. The table was compiled by randomly selecting 50 videos from the GLips 2022 dataset. Each TL model was employed as a feature extractor, and the number of parameters and the variance of the extracted features were reported.

Furthermore, we excluded ConvNextTiny from consideration due to its parameter count exceeding 20 million, which is higher than our threshold for model complexity. Additionally, EfficientNetB0 was not selected because its feature variance was less than 0.3 (specifically, 0.23), which did not meet our criteria for effective feature extraction. The detailed size of each model, as well as other pertinent characteristics, are presented in the table above for reference.

Table 2: Number of Parameters, Feature Extraction Variance, and Size of TL Models

TL Model	Parameters (#)	Variance	Model Size (MB)
VGG19	20M	0.34	549
DenseNet121	7M	2.10	33
MobileNetV3	3M	0.37	16
NASNetMobile	4M	1.12	23
ConvNextTiny	28M	1	109.42
EfficientNetB0	4M	0.23	29

1) NASNetMobile

NASNetMobile, an architecture in the form of a convolutional neural network (CNN), has been developed by Google. It was specifically designed for

efficient mobile applications and is part of Google's Neural Architecture Search (NAS) project. Utilizing a neural architecture search algorithm, NASNetMobile autonomously designs a CNN architecture optimized for mobile devices. This architecture comprises multiple convolutional layers with varying filter sizes, followed by max-pooling layers to down-sample the feature maps. Moreover, it incorporates skip connections to improve the flow of information between layers and mitigate the vanishing gradient issue. Despite its computational efficiency, NASNetMobile achieves high accuracy, making it suitable for deployment on devices with limited resources such as smartphones and embedded systems. Furthermore, NASNetMobile has been pre-trained on extensive image datasets like ImageNet, enabling it to learn complex visual features effectively.

Within the architecture of NASNetMobile, Cell structures play a pivotal role. These structures encompass two distinct types: Normal Cells (illustrated in Figure 1 (a)) and Reduction Cells (depicted in Figure 1 (b)). Comprising multiple convolutional layers with reusable and combinable characteristics, these Cell structures form sub-networks. NASNetMobile constructs its entire network by stacking these fundamental modules (Normal Cells and Reduction Cells) together. This modular approach grants NASNetMobile exceptional flexibility, enabling it to accommodate diverse task requirements and resource limitations. The network's architectural model is presented in Figure 1, featuring the optimal Normal Cell and Reduction Cell structures.

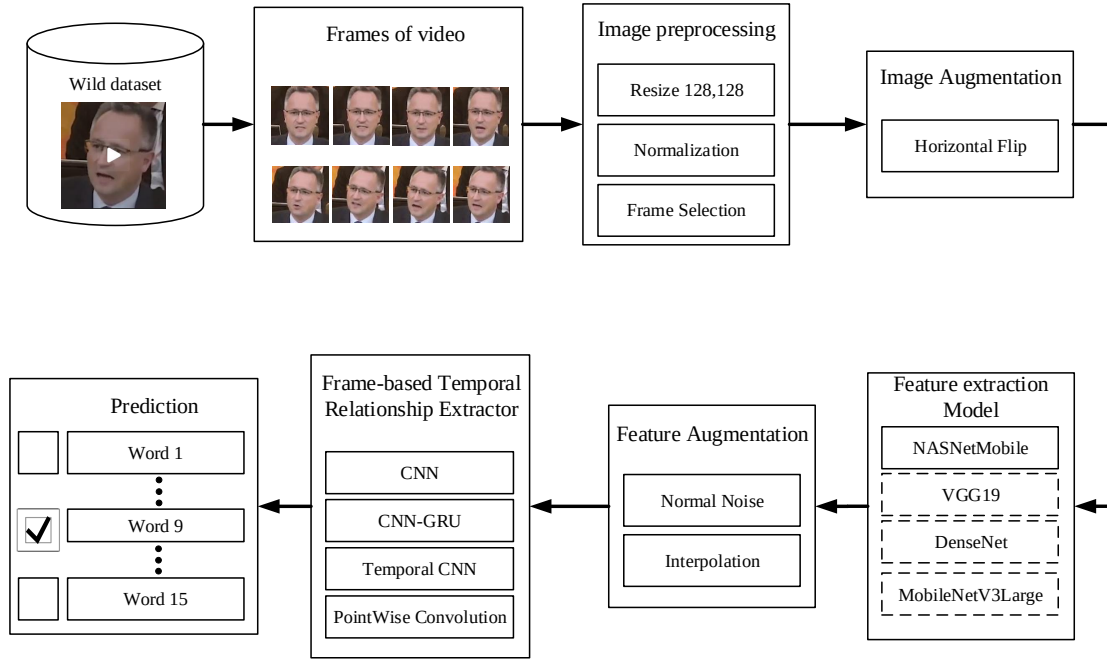


Figure 2: **Proposed framework for VSR by utilizing NASNetMobile fine-tuned via TL.**

2) VGG19

VGG19 is a convolutional neural network architecture created by the Visual Geometry Group (VGG) at the University of Oxford [36]. Comprising 19 layers, it includes 16 convolutional layers and three fully connected layers. Renowned for its simplicity and effectiveness in image classification tasks, VGG19 utilizes a structure characterized by repeated use of 3×3 convolutional filters, followed by max-pooling layers to diminish spatial dimensions. It has achieved notable performance in various computer vision tasks, particularly in image classification and feature extraction. Despite its depth and relatively large parameter count, VGG19 remains popular due to its straightforward architecture and strong performance on benchmark datasets like ImageNet.

3) DenseNet121

DenseNet121, known as Dense Convolutional Network, is a convolutional neural network design proposed by researchers at Facebook AI Research (FAIR) [11]. It introduces a distinctive connectivity pattern among layers, where each layer establishes connections with every other layer in a forward manner. This dense connectivity enables feature reuse across the network, enhancing the efficiency of gradient propagation and feature propagation across layers. In DenseNet121, each layer receives feature maps from all preceding layers and transmits its feature maps to all subsequent layers. This dense connectivity promotes feature propagation and en-

courages feature reuse, resulting in improved learning efficiency and model performance. The DenseNet121 architecture comprises dense blocks, transition layers, and a global average pooling layer followed by a softmax classifier. Dense blocks consist of multiple layers, each containing convolutional layers with batch normalization and ReLU activation functions. Transition layers are responsible for reducing the spatial dimensions of feature maps and controlling the number of channels before passing them to the next dense block. DenseNet121 has gained popularity in various computer vision tasks, such as image classification, object detection, and segmentation, due to its efficient parameter usage and robust performance.

4) MobileNetV3Large

MobileNetV3Large is a variant of the MobileNetV3 architecture, specifically tailored for efficient deployment on mobile and embedded devices [9]. It is optimized to strike a balance between model size, accuracy, and computational efficiency, making it well-suited for resource-constrained environments. MobileNetV3Large incorporates various design features to excel in tasks like image classification and object detection while keeping computational demands minimal.

The architecture of MobileNetV3Large includes depthwise separable convolutions, which effectively decompose standard convolutions into depthwise and pointwise convolutions. This reduction in parameters and computational overhead maintains the net-

work's representational capacity. Furthermore, MobileNetV3Large introduces efficient inverted residuals with linear bottlenecks, employing lightweight bottleneck structures to further enhance performance.

D. FRAME-BASED TEMPORAL RELATIONSHIP EXTRACTOR

In VSR applications, a vital part of the methodology entails scrutinizing the complex relationships among frames in a video sequence. This follows the initial feature extraction from each frame. The primary goal is to understand the nuanced connections of each frame, ultimately comprehending spoken words. This understanding relies on analyzing the subtle cues provided by lip movements observed throughout the video, extracted during the feature extraction phase. These methods include Convolutional Neural Networks (CNNs), CNN-Gated Recurrent Units (CNN-GRU), Temporal CNN, and Temporal PointWise architectures.

Each of these methods plays a distinct role in our framework. CNNs handle initial spatial feature extraction, while CNN-GRU and Temporal CNN methods integrate temporal dynamics crucial for accurate recognition of visual speech patterns. Temporal PointWise further refines temporal relationships at a finer granularity, enhancing the model's ability to discern subtle variations in speech dynamics.

By detailing these feature extraction methods, we underscore their collective contribution to enhancing the framework's robustness and effectiveness in VSR tasks. This comprehensive approach ensures that our model can effectively capture and utilize both spatial and temporal information inherent in video data, thereby improving overall performance in real-world scenarios.

1) GRU

Gated recurrent units (GRU), a type of recurrent neural network (RNN) architecture, was developed by Kyunghyun Cho et al. [4] to address the vanishing gradient problem common in traditional RNNs. It serves as a simpler alternative to the Long Short-Term Memory (LSTM) architecture. Comprising update and reset gates, GRU regulates information flow within the network. Its key feature is capturing long-term dependencies in sequential data while remaining computationally efficient. Unlike LSTM, which uses separate memory cells and hidden states, GRU combines these into a single state vector, simplifying the architecture. GRU is popular in tasks like natural language processing, speech recognition, and time series prediction, known for its effectiveness with fewer parameters and faster training compared to LSTM.

2) Temporal CNN

Temporal CNNs leverage one-dimensional convolutional layers to analyze temporal patterns in sequential data [18]. They retain the core principles of traditional CNNs, including convolutional layers, activation functions, and pooling layers. However, they are tailored to process sequential data by applying one-dimensional convolutional operations to input sequences. This adaptation allows Temporal CNNs to effectively capture temporal dependencies and patterns within the data, eliminating the need for manual feature engineering. The network architecture typically consists of multiple convolutional layers followed by activation functions such as ReLU and pooling layers for dimensionality reduction. These components are often organized in a stacked manner to construct deep networks capable of capturing increasingly abstract temporal features.

3) Temporal PoinWise

Temporal PoinWise, also known as 1x1 convolution, is a frequently used operation in neural network architectures, especially in convolutional neural networks (CNNs) [10]. Unlike traditional convolutional operations that involve kernels with spatial dimensions (such as 3x3 or 5x5), pointwise convolution operates exclusively on the channel dimension of the input tensor. During pointwise convolution, each output channel is computed by taking a weighted sum of the corresponding input tensor channels, with the weights being adjusted during the training process, this operation can be represented as follows:

$$\text{output}(x, y, k) = \sum_{i=1}^{C_{in}} \text{input}(x, y, i) \times \text{kernel}(1, 1, i, k)$$

where:

- $\text{output}(x, y, k)$ is the value of the k -th output channel at spatial location (x, y) .
- $\text{input}(x, y, i)$ is the value of the i -th input channel at spatial location (x, y) .
- $\text{kernel}(1, 1, i, k)$ represents the weight associated with the i -th input channel and the k -th output channel.

IV. PROPOSED METHOD

In this section, we present our proposed method in two parts: the proposed framework, which outlines the main project steps through a diagram, and the proposed TL architecture, which details the NN architecture of our model.

A. PROPOSED FRAMEWORK

Our objective is to devise a novel model that attains suitable accuracy for the TL model in VSR on the wild dataset. Figure 2 presents the diagram of

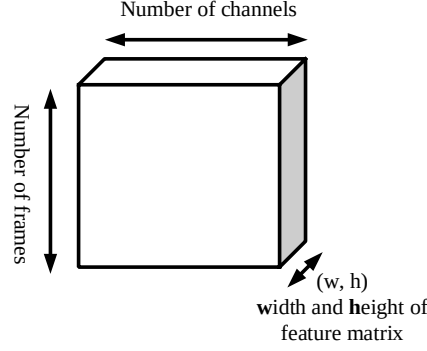


Figure 3: Sample of block diagram in our proposed architecture of model.

our proposed model aimed at achieving this goal. Our proposed model involves several stages: initially, each input data is converted into frames of its corresponding video, followed by image preprocessing and augmentation. Then, a TL model is employed to extract features, which are further enhanced using augmentation techniques. Subsequently, frame-based temporal analysis is applied to establish relationships between video frames, and finally, classification is performed to predict the spoken word based on the speaker's lip movements.

We utilized the GLips 2022 dataset as our input data, as extensively detailed in Section III-A. Initially, the videos underwent conversion into individual frames to enable subsequent analysis, as outlined in Section III-B. These frames were then subjected to image preprocessing techniques to enhance their suitability for model training, including resizing to 128×128 pixels for standardization across all frames. Subsequently, normalization techniques, as discussed in Section III-B1, were applied to ensure consistent pixel intensity values. To optimize computational efficiency and focus on relevant temporal information, a subset of 16 frames was selected from each video for further analysis. This methodical approach ensured the appropriate preparation and optimization of our input data for subsequent modeling stages.

We employed two augmentation techniques, as explained in Section III-B2. Firstly, a horizontal flip was applied to the images extracted from the videos. Secondly, feature augmentation was performed on the features extracted from each image using the TL model. This process included adding Normal noise, as previously discussed, and utilizing interpolation techniques outlined in Section III-B2.

Feature extraction in this project comprises two phases. Initially, a TL model is employed to extract features from each frame of the video data. Subsequently, these features are utilized to infer temporal relationships between the video frames. In Section III-C, we discuss the models NASNetMobile,

VGG19, DenseNet, and MobileNetV3Large, used for feature extraction. NASNetMobile is used in our proposed model, while the others are baseline models. Each baseline model undergoes individual evaluation against our proposed model within the same framework, as shown in Figure 2 in the frame extraction section, indicated by dashed lines. Additionally, the CNN-GRU, CNN, Temporal CNN, and Temporal PointWise models, detailed in Section III-D, are utilized in the frame-based temporal relationship extraction phase.

In our methodology, we focused on enhancing VSR by leveraging a novel architecture that extract features of video frames using NASNet. We selected NASNet among various TL models based on its superior empirical performance. NASNet features numerous layers with a low number of parameters per layer, which enhances computational speed and reduces both complexity and processing time. This design makes NASNet particularly suitable for mobile applications, aligning with our study's primary objective of developing an efficient and practical model for real-world use.

B. PROPOSED TL ARCHITECTURE

In this section, we present the proposed neural network (NN) architecture for our proposed model, featuring four different backend shapes: CNN, CNN-GRU, Temporal CNN, and Temporal PoinWise. Figure 4 illustrates the four architectures: (a) CNN, (b) CNN-GRU (Section III-D1), (c) Temporal CNN (Section III-D2), and (d) Temporal PoinWise (Section III-D3). All the Layers in these blocks include L2 norm regularization. All block diagrams in figures 4 adhere to the same format as illustrated in figure 5, indicating the number of frames, channels, and dimensions of the feature matrix in each state through accompanying notes.

After preprocessing the input video, it undergoes feature extraction using TL models during the front-end phase of VSR, as outlined in section III-B. Figure

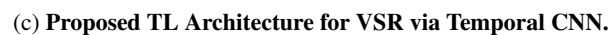
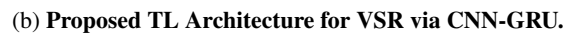


Figure 4: The design of the proposed TL architecture for VSR.

Table 3: Impact of augmentation techniques on NASNetMobile Model with CNN. The table compares models with no augmentation, image augmentation (vertical flip), and combinations of image and feature augmentation (normal noise and interpolation). Performance metrics include categorical loss function, accuracy, F1 score, and weighted F1 score.

Model	Categorical Loss Function	Accuracy	F1 Score	Weighted F1 Score
No Augmentation	5.3878	0.1480	0.1458	0.1458
Image Augmentation (Vertical Flip)	3.3063	0.3653	0.36.5	0.3605
Image Augmentation (Vertical Flip) + Feature Augmentation (Normal Noise)	3.093	0.3853	0.3806	0.3806
Image Augmentation (Vertical Flip) + Feature Augmentation (Normal Noise + Interpolation)	2.7742	0.4840	0.4850	0.4850

4 shown each TL model produces a 4×4 feature matrix representing 16 frames, with MobileNetV3Large, NASNet, Dense, and VGG19 yielding channel sizes of 960, 1056, 1024, and 576, respectively. After feature extraction, batch normalization is applied to maintain the original feature size. Then, PointWise convolution 2D is used to ensure a consistent channel size of 128 for each TL model and also another batch normalization block.

In Figure 4 (a), our proposed CNN model architecture involves strided convolution 2D, followed by ReLU activation and strides (2, 2), to reduce feature dimensions to 2×2 feature matrix with 128 channels. After preprocessing, batch normalization is applied, followed by transformation into fully connected layers with dropout, resulting in a size of $8,192 \times 1$. The output layer is then configured for multi-classification. In Figure 4 (b), after strided convolution and batch normalization, we introduce GRU to assess its impact on temporal analysis. GRU was chosen for this purpose. Prior to GRU application, the feature tensor is reshaped from $128 \times 2 \times 2$ to $512 \times 1 \times 1$. Subsequently, two GRU layers are employed, each followed by batch normalization blocks sized $64 \times 1 \times 1$. Finally, fully connected layers and dropout prepare the output. In Figure 4 (c), we utilize two dilated convolutions with ReLU activation, similar to the CNN-GRU setup. These convolutions operate at two different dilation rates ($d=1$ and $d=2$) to capture information from multiple frames up to the second step. This approach replaces the use of GRU as depicted in Figure 4 (b). In the final NN architecture, we skip the strided convolution used in previous models and opt for temporal convolution instead. This setup comprises three 2D dilated convolutions with ReLU activation, each with dilation rates of 1, 2, and 4. The convolution blocks have dimensions of $128 \times 1 \times 1$, $128 \times 1 \times 1$, and $96 \times 1 \times 1$, respectively, along with identical batch normalization blocks. Prior to these convolutions, we apply a reshape block to resize the input to $2048 \times 1 \times 1$ dimensions.

V. ABLATION STUDY

To understand the contribution of each component in our proposed framework, we conducted a comprehensive ablation study. Figure 2 introduces our proposed framework for VSR, which includes several critical steps: image preprocessing, image augmentation, feature augmentation, and frame-based temporal relationship extraction.

The rationale behind our component analysis is to evaluate the impact of each significant step in our framework. This includes, Image Preprocessing which is a necessary process that cannot be skipped in the ablation study as it ensures consistent scaling of dataset features. Image Augmentation, this step prepares a diverse range of images for feature extraction. In the first step of our ablation study, we skip image augmentation to assess its impact. Feature Augmentation: This step prepares the features of frames for the frame-based temporal relationship extractor. Our second and third steps of the ablation study involve skipping each component of feature augmentation to evaluate their contributions. We systematically conducted experiments by removing or modifying each component of our framework. Each configuration was evaluated using the same dataset and performance metrics to ensure consistency. The specific setups included:

- Evaluating the framework without image augmentation.
- Evaluating the framework without feature augmentation.
- Evaluating the framework with both image and feature augmentation omitted.

Our results in table 3 indicated that each component plays a significant role in the overall performance of the model. Without Image Augmentation, the performance metrics showed a noticeable decrease in accuracy, indicating the importance of preparing a diverse range of training images. Without Feature Augmentation, there was a significant drop in the model's ability to generalize, highlighting

the necessity of feature augmentation for extracting meaningful temporal relationships. Without Both Augmentations, the combined omission led to the poorest performance, confirming that both image and feature augmentation are crucial for achieving robust results.

VI. EXPERIMENT

In this section, we outline three types of experiments. First, we introduce the experimental setup, followed by an evaluation of our model's classification performance. Next, we compare TL models, and finally, we conduct a comparative analysis with existing models.

A. EXPERIMENTAL SETTING

The GLips 2022 dataset, as detailed in Section III-A, served as the foundation for evaluating our model as described in Section IV. Our experimental setup entailed implementing our methodology using Python with the TensorFlow package on a system featuring a GPU T4, an Intel Xeon CPU with two virtual CPUs, and 12 GB of RAM.

For all proposed transfer learning (TL) strategies, a batch size of 32 was employed. GLips videos, which have a frame rate of 29 to 30 frames per second, were padded to 32 frames to match the batch size requirements. The frames were resized to 128x128 pixels. A learning rate of 0.001 was set, and the models were trained for 200 epochs using the Categorical Cross-Entropy loss function and the Adam optimizer. These procedures were consistently applied to all models illustrated in Figure 4. Default hyperparameters were used to focus on comparing the TL models without evaluating the effects of hyperparameter variations.

The test, validation, and train sets of this dataset were pre-defined, ensuring consistency across diverse model architectures. Evaluation metrics included accuracy, categorical cross-entropy, F1 score, weighted F1 score, and confusion matrix.

- Accuracy measures the proportion of correct predictions out of the total predictions made. It is calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

- Categorical cross-entropy quantifies the disparity between predicted probabilities ($\hat{y}$) and actual class labels (y). It is defined as:

$$\text{Categorical cross-entropy} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$$

where N is the number of instances, C is the number of classes, y_{ij} is the indicator function for class j in instance i , and $\hat{y}_{ij}$ is the predicted probability for class j in instance i .

- The F1 score is the harmonic mean of precision (P) and recall (R). It is computed as:

$$\text{F1 score} = \frac{2 \times P \times R}{P + R}$$

where precision (P) and recall (R) are given by:

$$P = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

$$R = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- The Weighted F1 score extends the F1 score to account for class imbalances by computing the average F1 score weighted by the number of instances in each class.

The confusion matrix summarizes the model's performance and is represented as:

$$\begin{bmatrix} \text{True Negatives} & \text{False Positives} \\ \text{False Negatives} & \text{True Positives} \end{bmatrix}$$

1) Experiment 1: Classification performance

In this experiment, we assess the classification performance of the model introduced in Section IV, shown in Figure 2, across four different architectures outlined in Figure 4.

Table 4 presents the performance evaluation of VGG19, DenseNet121, MobileNetV3Large, and NASNetMobile within the proposed framework. These models were tested using four different architectures: CNN, CNN-GRU, Temporal CNN, and Temporal PoinWise. The best performance metrics for each model are highlighted in bold. The CNN architecture consistently achieves the highest accuracy, F1 score, and weighted F1 score across all models. Additionally, the CNN architecture achieves the best loss function performance, except for MobileNetV3Large, where the Temporal CNN architecture performs better. This analysis demonstrates the overall effectiveness of the CNN architecture in the proposed framework, with noted exceptions.

The table shows the statistically significant improvements of NASNetMobile over the VGG19, DenseNet121, and MobileNetV3Large models. Improvements are marked with $\star$ for VGG19, $\dagger$ for DenseNet121, and $\ddagger$ for MobileNetV3Large. These improvements are statistically significant when the T-test P-value is less than 0.05, indicating NASNetMobile's superior performance.

Figure 5 shows the performance of the proposed model using a confusion matrix, as outlined in Section VI-A. The confusion matrix is a valuable tool for assessing classification model performance by indicating how well predictions match actual classes. The diagonal elements represent true positives (TP). High values (near one, shown in blue) in these cells

Table 4: Classification Performance of VGG19, DenseNet121, MobileNetV3Large, and NASNetMobile in proposed model by using CNN, CNN-GRU, Temporal CNN, and Temporal PointWise in proposed architecture. Statistically significant improvements over VGG19, DenseNet121, and MobileNetV3Large are shown by *, †, and ‡, respectively.

(a) VGG19				
Model Architecture	Categorical Loss function	Accuracy	F1 score	Weighted F1 score
CNN	2.3188	0.4507	0.4461	0.4461
CNN-GRU	3.0907	0.3920	0.3860	0.3860
Temporal CNN	2.9133	0.3827	0.3825	0.3825
Temporal PointWise	2.7533	0.4187	0.41937	0.4197
(b) DenseNet121				
CNN	2.7100	0.4747	0.4724	0.4724
CNN-GRU	2.8951	0.4347	0.4347	0.4347
Temporal CNN	2.9300	0.4160	0.4157	0.4157
Temporal PointWise	3.1634	0.4053	0.4056	0.4056
(c) MobileNetV3Large				
CNN	4.8642	0.1387	0.1316	0.1316
CNN-GRU	4.1797	0.1013	0.0798	0.0798
Temporal CNN	4.0404	0.1267	0.1144	0.1144
Temporal PointWise	4.8227	0.1320	0.109662	0.1096
(d) NASNetMobile				
CNN	2.7742 *‡	0.4840 ‡	0.4850 ‡	0.4850 ‡
CNN-GRU	3.0645 *‡	0.4173 *†‡	0.4226 *†‡	0.4226 *†‡
Temporal CNN	3.1268 ‡	0.4093‡	0.40711 ‡	0.4071 ‡
Temporal PointWise	3.3841 *†‡	0.3880 *†‡	0.3877 *†‡	0.3878 *†‡

signify that the model is performing well. For instance, values of 50 for Class 0 and 45 for Class 1 indicate correct predictions where the predicted class matches the actual class. Specifically, for the class "digitalisierung," a value of 0.64 represents correct predictions. The off-diagonal elements represent misclassifications, including false positives and false negatives. For example, in the first row, a value of 0.02 in the Predicted label column shows that 0.02 instances of Class "aber" were misclassified as Class "aufgaben".

B. EXPERIMENT 2: COMPARING TL MODELS PERFORMANCE

In this experiment, we compare the performance of four TL models: VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile. These models are evaluated using three metrics: accuracy, F1 score, and weighted F1 score. The results of our experiments are shown in two figures. Figure 6 illustrates the best performance of each model with CNN backend over 200 epochs, according to the experimental settings detailed in Section VI-A. Figure 7 presents the val-

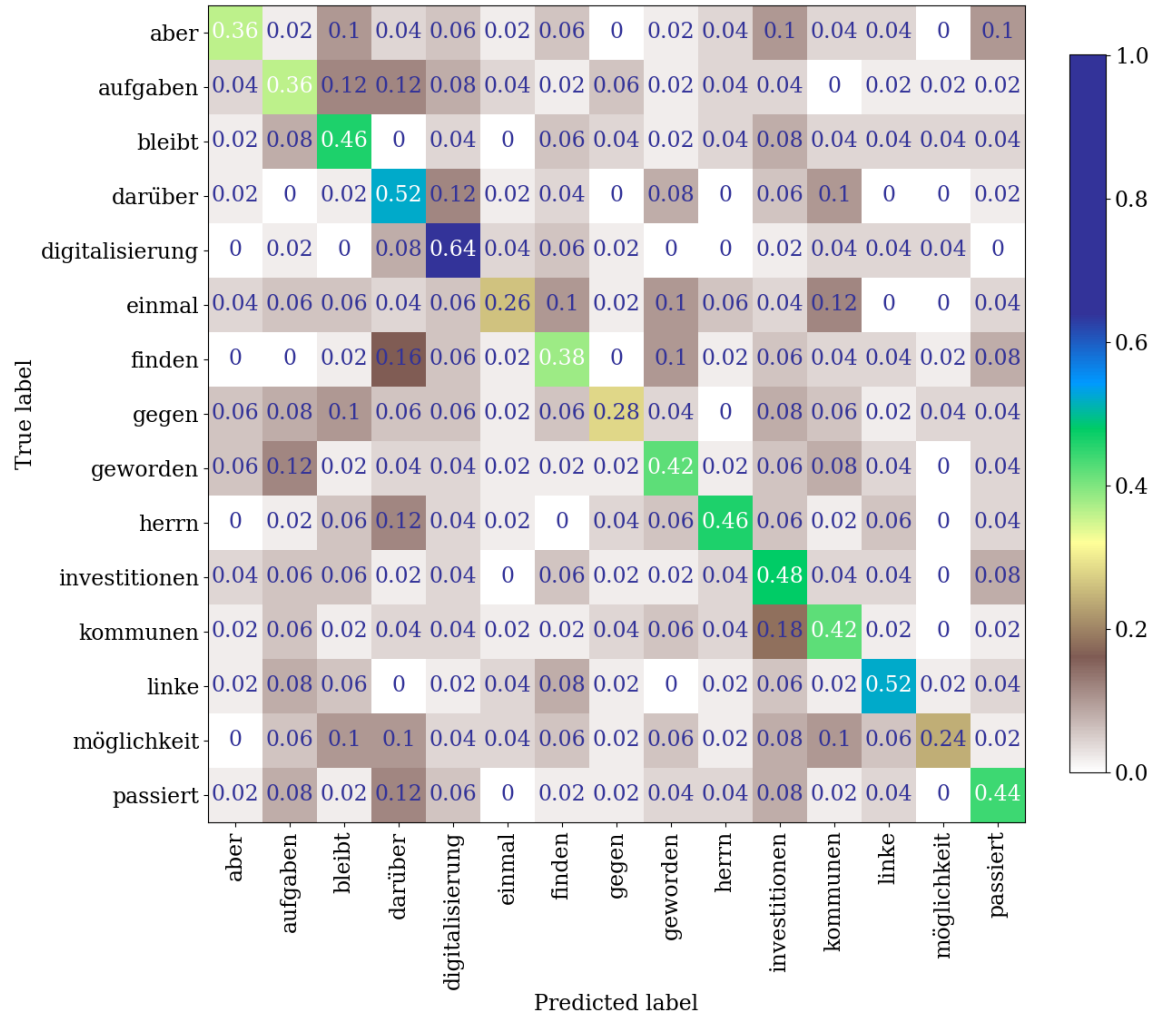


Figure 5: Confusion Matrix of NASNetMobile on GLips 2022 for 15 words classification.

idation set measurements recorded during the model training process.

Figure 6 shows the highest values of accuracy, F1 score, and Weighted F1 score for VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile. These metrics were evaluated across four architectures: CNN, CNN-GRU, Temporal CNN, and Temporal PoinWise in Experiment VI-A1. Figure 6 indicates that NASNetMobile outperformed the other models in classifying 15 words in the VSR system in CNN architecture.

Figure 7 shows the validation set metrics, including categorical loss, accuracy, F1 score, and weighted F1 score, for VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile models trained with the CNN architecture. These metrics provide

a comprehensive evaluation of model performance, allowing for a detailed comparison of each model's effectiveness within the CNN framework. Categorical loss measures classification errors, accuracy indicates overall prediction correctness, and the F1 and weighted F1 scores offer insights into the balance between precision and recall, and performance across different classes. As this figure shows the NASNet-Mobile model gets the high performance among another model.

C. EXPERIMENT 3: COMPARATIVE ANALYSIS

In this experiment, we compare TL models by evaluating their parameter counts and performance metrics. As previously mentioned, NASNetMobile achieved the best performance in Experiment VI-B.

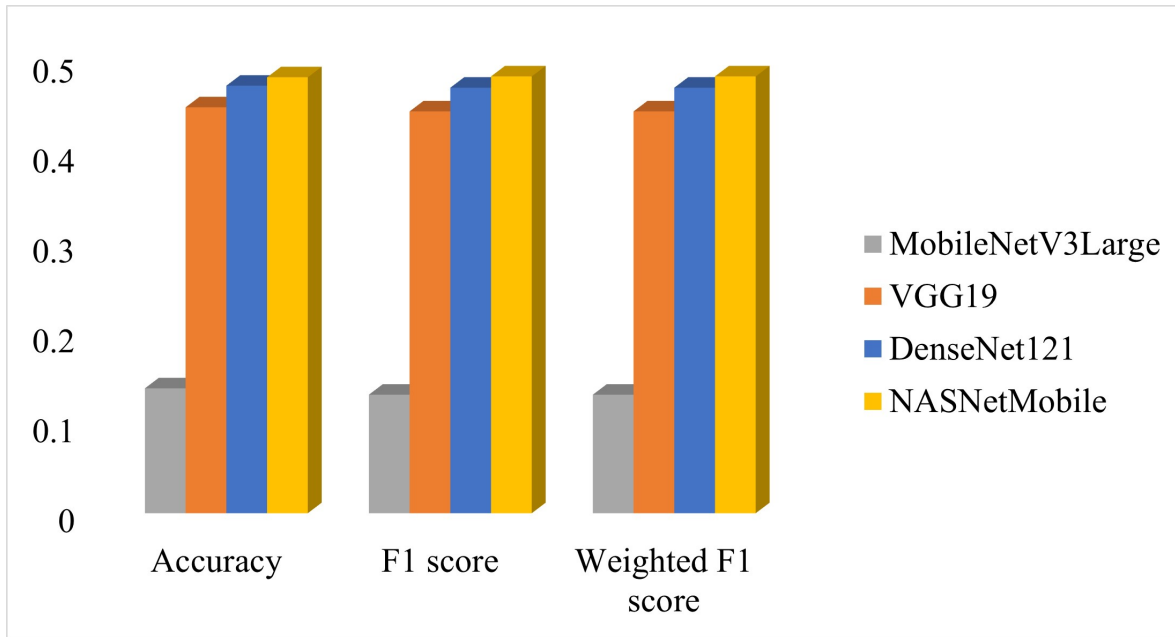


Figure 6: Comparing TL models performanc on GLips 2022 for 15 words classification by CNN architecture.

Here, we analyze the parameter counts of each model to understand their impact on performance. This analysis will reveal the trade-offs between model complexity and effectiveness, providing insights into optimal configurations for TL models.

Figure 8 demonstrates that NASNetMobile not only achieves the highest performance but also has the lowest parameter count, making it the most efficient TL model. The X-axis shows the number of parameters for each TL model within the CNN architecture, as detailed in Figure 4 (part a). The Y-axis presents the metrics for each model: (a) categorical loss function, (b) accuracy, (c) F1 score, and (d) weighted F1 score. All models were evaluated using the same validation sets and settings described in Section VI-A. The size of each bubble indicates the parameter count for each model. NASNetMobile consistently demonstrates superior performance across all metrics relative to its parameter count.

In our study, we employed the NASNetMobile model, which features a deep architecture with fewer parameters per layer. This design choice effectively reduces computational demands while maintaining high performance. This efficiency is particularly beneficial for real-time applications and deployment on mobile devices. For VSR tasks, rapid and accurate video data processing is crucial. By using NASNet-Mobile, we ensure that the VSR system functions effectively even in resource-limited environments, making it a practical solution for mobile applications.

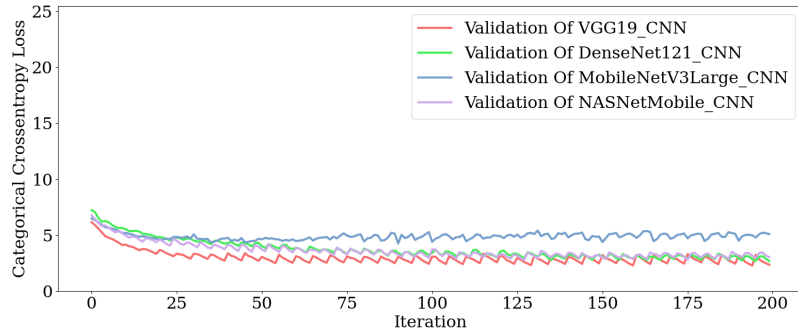
VII. DISCUSSION, CONCLUSION AND FUTURE WORK

This study addresses challenges posed by wild datasets in VSR using a novel framework based on TL models. High performance was achieved with the lightweight NASNetMobile model, aiming to overcome two primary gaps: the impracticality of heavy models for mobile applications due to high computational demands, and the reliance on synthetic data for testing, which fails to represent real-world scenarios accurately.

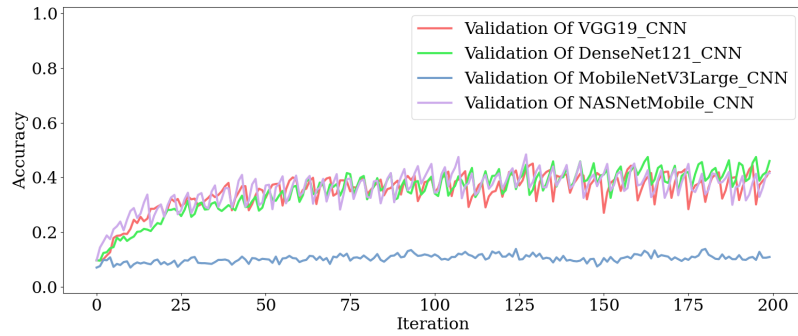
The proposed framework integrates preprocessing, data augmentation, and TL models for feature extraction from video data. TL models utilized include VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile, crucial for extracting temporal features via CNN, CNN-GRU, Temporal CNN, and Temporal PointWise techniques. These methods form the core of the model, facilitating robust feature extraction for VSR tasks.

Preprocessing involves normalization for consistent feature scaling and augmentation techniques such as horizontal flipping, noise addition, and interpolation to enhance dataset variability. The framework consists of two phases: feature extraction using TL models and frame-based temporal relationship extraction.

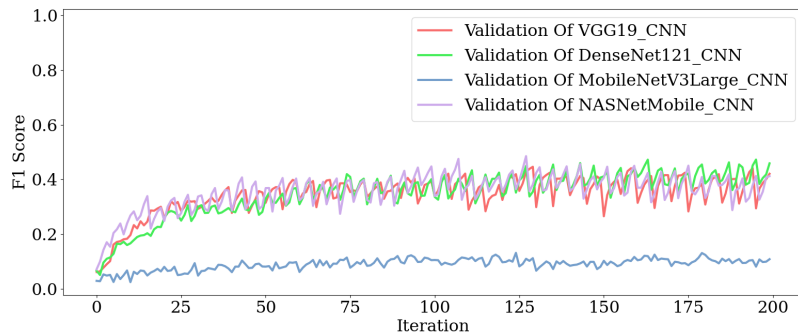
Four TL architectures—CNN, CNN-GRU, Temporal-CNN, and Temporal PointWise—were proposed and evaluated using pretrained VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile models. Evaluation metrics include loss function,



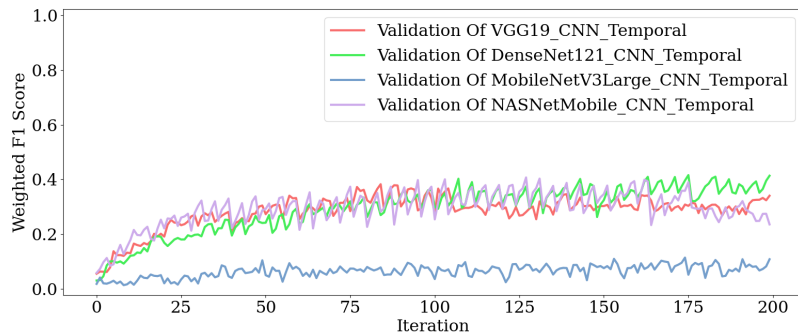
(a) Categorical Loss function.



(b) Accuracy.



(c) F1 Score.



(d) W F1 Score.

Figure 7: Validation Set Metrics: Loss Function, Accuracy, F1 Score, and Weighted F1 Score During Training of VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile Models Using CNN Architecture.

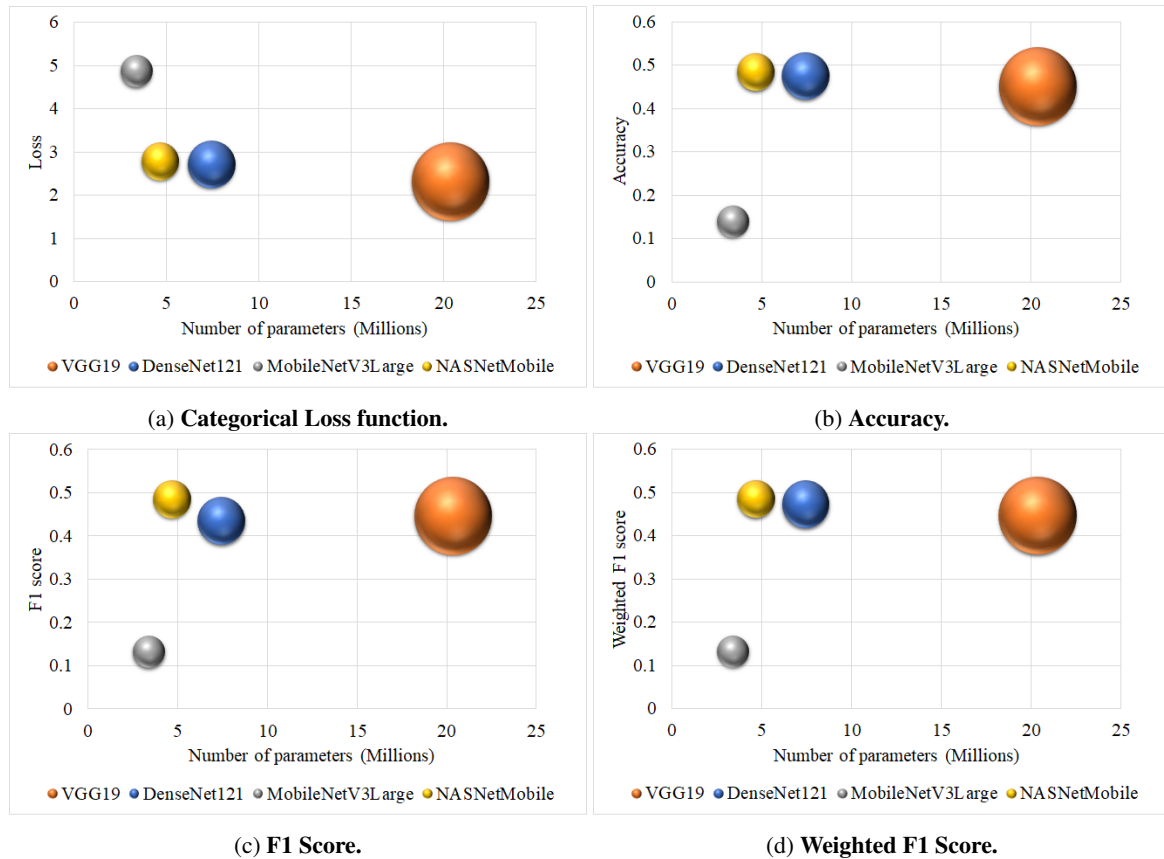


Figure 8: Comparison of VGG19, MobileNetV3Large, DenseNet121, and NASNetMobile CNN architectures based on parameter count and performance metrics, including Loss Function, Accuracy, F1 Score, and Weighted F1 Score.

accuracy, F1 score, and weighted F1 score. NASNetMobile, with 4,680,739 parameters, achieved an accuracy of 0.4840, an F1 score of 0.4850, and a weighted F1 score of 0.4850, demonstrating superior performance over other models.

The findings from the ablation study underscore the importance of each component in our VSR framework. The empirical validation demonstrated that image preprocessing, augmentation, and feature augmentation are all vital for enhancing the model's accuracy and generalization capabilities. NASNet's architecture, when combined with these steps, provides a balanced trade-off between computational efficiency and performance, making it suitable for real-time applications in lip reading.

Experimental results indicate our model outperforms VGG19, MobileNetV3Large, and DenseNet121 with reduced weight and complexity. However, challenges remain in improving prediction accuracy due to data insufficiency and labeling uncertainties in wild datasets. NASNet is identified as the optimal TL model for feature extraction, and CNN excels as the TL architecture for frame-based temporal relationships in VSR.

Trained on the GLips 2022 dataset, future research should expand evaluation to additional datasets for comprehensive validation. Comparative analyses with other TL models and exploring training without transfer learning offer avenues for further insights and improvements.

References

- [1] Madina Abdrakhmanova et al. "Speaking-faces: A large-scale multimodal dataset of voice commands with visual and thermal video streams". In: *Sensors* 21.10 (2021), p. 3465.
- [2] Mohammed Abid Abrar et al. "Deep lip reading-a deep learning based lip-reading software for the hearing impaired". In: *2019 IEEE R10 Humanitarian Technology Conference (R10-HTC)*(47129). IEEE. 2019, pp. 40–44.
- [3] Tyler Benster et al. "A Cross-Modal Approach to Silent Speech with LLM-Enhanced Recognition". In: *arXiv preprint arXiv:2403.05583* (2024).
- [4] Kyunghyun Cho et al. "Learning phrase representations using RNN encoder-decoder for sta-

- tistical machine translation". In: *arXiv preprint arXiv:1406.1078* (2014).
- [5] J Chung and Andrew Zisserman. "Lip reading in profile". In: *British Machine Vision Conference, 2017*. British Machine Vision Association and Society for Pattern Recognition. 2017.
- [6] Penghao Dong et al. "Decoding silent speech commands from articulatory movements through soft magnetic skin and machine learning". In: *Materials Horizons* 10.12 (2023), pp. 5607–5620.
- [7] Themis Exarchos et al. "Lip-Reading Advancements: A 3D Convolutional Neural Network/Long Short-Term Memory Fusion for Precise Word Recognition". In: *BioMedInformatics* 4.1 (2024), pp. 410–422.
- [8] Dalu Feng et al. "Audio-driven deformation flow for effective lip reading". In: *2022 26th International Conference on Pattern Recognition (ICPR)*. IEEE. 2022, pp. 274–280.
- [9] Andrew Howard et al. "Searching for mobilenetv3". In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 1314–1324.
- [10] Binh-Son Hua, Minh-Khoi Tran, and Sai-Kit Yeung. "Pointwise convolutional neural networks". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 984–993.
- [11] Gao Huang et al. "Densely connected convolutional networks". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 4700–4708.
- [12] Hongyang Huang et al. "A novel machine lip reading model". In: *Procedia Computer Science* 199 (2022), pp. 1432–1437.
- [13] Denis Ivanko et al. "Visual speech recognition in a driver assistance system". In: *2022 30th European Signal Processing Conference (EU-SIPCO)*. IEEE. 2022, pp. 1131–1135.
- [14] Tatsuya Kanamaru and Takeshi Saitoh. "KuchiNavi: lip-reading-based navigation app". In: *Fifteenth International Conference on Graphics and Image Processing (ICGIP 2023)*. Vol. 13089. SPIE. 2024, pp. 460–466.
- [15] Minsu Kim, Jeong Hun Yeo, and Yong Man Ro. "Distinguishing homophones using multi-head visual-audio memory for lip reading". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 36. 1. 2022, pp. 1174–1182.
- [16] Kshitiz Kumar, Tsuhan Chen, and Richard M Stern. "Profile view lip reading". In: *2007 IEEE International Conference on Acoustics, Speech and Signal Processing-ICASSP'07*. Vol. 4. IEEE. 2007, pp. IV–429.
- [17] Hendrik Laux et al. "Two-stage visual speech recognition for intensive care patients". In: *Scientific Reports* 13.1 (2023), p. 928.
- [18] Colin Lea et al. "Temporal convolutional networks: A unified approach to action segmentation". In: *Computer Vision—ECCV 2016 Workshops: Amsterdam, The Netherlands, October 8–10 and 15–16, 2016, Proceedings, Part III* 14. Springer. 2016, pp. 47–54.
- [19] Muxin Liao et al. "Calibration-based Dual Prototypical Contrastive Learning Approach for Domain Generalization Semantic Segmentation". In: *Proceedings of the 31st ACM International Conference on Multimedia*. 2023, pp. 2199–2210.
- [20] Muxin Liao et al. "Exploring more concentrated and consistent activation regions for cross-domain semantic segmentation". In: *Neurocomputing* 500 (2022), pp. 938–948.
- [21] Zhijie Lin et al. "Simullr: Simultaneous lip reading transducer with attention-guided adaptive memory". In: *Proceedings of the 29th ACM International Conference on Multimedia*. 2021, pp. 1359–1367.
- [22] Pingchuan Ma, Stavros Petridis, and Maja Pantic. "Visual Speech Recognition for Multiple Languages". In: ().
- [23] M Musalia et al. "A user evaluation of speech/phrase recognition software in critically ill patients: a DECIDE-AI feasibility study". In: *Critical Care* 27.1 (2023), p. 277.
- [24] Praneeth Nemani et al. "Deep learning based holistic speaker independent visual speech recognition". In: *IEEE Transactions on Artificial Intelligence* (2022).
- [25] Dharin Parekh et al. "Lip reading using convolutional auto encoders as feature extractor". In: *2019 IEEE 5th International Conference for Convergence in Technology (I2CT)*. IEEE. 2019, pp. 1–6.
- [26] Javad Peymanfard et al. "Word-level Persian lipreading dataset". In: *2022 12th International Conference on Computer and Knowledge Engineering (ICCKE)*. IEEE. 2022, pp. 225–230.
- [27] KR Prajwal, Triantafyllos Afouras, and Andrew Zisserman. "Sub-word level lip reading with visual attention". In: *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*. 2022, pp. 5162–5172.
- [28] SJ Preethi et al. "Analyzing lower half facial gestures for lip reading applications: Survey on vision techniques". In: *Computer Vision and Image Understanding* (2023), p. 103738.

- [29] Mohammad Hasan Rahmani and Farshad Almasganj. "Lip-reading via a DNN-HMM hybrid system using combination of the image-based and model-based features". In: *2017 3rd International Conference on Pattern Recognition and Image Analysis (IPRIA)*. IEEE. 2017, pp. 195–199.
- [30] Gerald Schwiebert et al. "A multimodal German dataset for automatic lip reading systems and transfer learning". In: *arXiv preprint arXiv:2202.13403* (2022).
- [31] Changchong Sheng et al. "Deep learning for visual speech analysis: A survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
- [32] Changchong Sheng et al. "Importance-aware information bottleneck learning paradigm for lip reading". In: *IEEE Transactions on Multimedia* (2022).
- [33] Ali Jafari Sheshpoli and Ali Nadian-Ghomsheh. "Temporal and spatial features for visual speech recognition". In: *Fundamental Research in Electrical Engineering: The Selected Papers of The First International Conference on Fundamental Research in Electrical Engineering*. Springer. 2019, pp. 135–145.
- [34] Brendan Shillingford et al. "Large-scale visual speech recognition". In: *arXiv preprint arXiv:1807.05162* (2018).
- [35] Jayanth Shree Kumar et al. "Improved viseme recognition using generative adversarial networks". In: *2020 IEEE REGION 10 CONFERENCE (TENCON)*. IEEE. 2020, pp. 1118–1123.
- [36] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).
- [37] Ganchao Tan et al. "Multi-grained spatio-temporal features perceived network for event-based lip-reading". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 20094–20103.
- [38] Mingxing Tan et al. "Mnasnet: Platform-aware neural architecture search for mobile". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 2820–2828.
- [39] Kwanchiva Thangthai. "Computer lipreading via hybrid deep neural network hidden Markov models". PhD thesis. University of East Anglia, 2018.
- [40] C Wang. "Multi-grained spatio-temporal modeling for lip-reading. arXiv 2019". In: *arXiv preprint arXiv:1908.11618* ().
- [41] Xue Wang et al. "Watch Your Mouth: Silent Speech Recognition with Depth Sensing". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 2024, pp. 1–15.
- [42] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. "A survey of transfer learning". In: *Journal of Big data* 3 (2016), pp. 1–40.
- [43] Jingyun Xiao et al. "Deformation flow based two-stream network for lip reading". In: *2020 15th IEEE international conference on automatic face and gesture recognition (FG 2020)*. IEEE. 2020, pp. 364–370.
- [44] Xueyi Zhang et al. "Boosting lip reading with a multi-view fusion network". In: *2022 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE. 2022, pp. 1–6.
- [45] Haoran Zhao, Bowen Zhang, and Zhanhang Yin. "Lip-Corrector: Application of BERT-based model in sentence-level lipreading". In: *Journal of Physics: Conference Series*. Vol. 1871. 1. IOP Publishing. 2021, p. 012146.
- [46] Ya Zhao et al. "Speech guided disentangled visual representation learning for lip reading". In: *Proceedings of the 2021 International Conference on Multimodal Interaction*. 2021, pp. 687–691.



ASHWAQ WALEED ABDUL AMEER a teacher at the Faculty of Engineering - Imam Al-Kadhim College of Sciences, Department of Computer Technology Engineering. I obtained a bachelor's degree in computer technology engineering from Al-Rafidain University College in Baghdad in 2013, and obtained a master's degree in computer engineering in 2017 from Al-Razi University in Kermanshah in the Islamic Republic of Iran, and now I am a PhD student at the University of Tabriz also in Iran, the field of scientific research is Internet of things, application design, websites, automation systems, and artificial intelligence applications. machine learning.



PEDRAM SALEHPOUR He received his B.Sc. in Computer Hardware from University of Tabriz, Tabriz, Iran. He received his MSc. In computer science and Ph.D. in electrical engineering from the University of Tabriz. He later joined Electrical and Computer Engineering Faculty of University of Tabriz. His research interests include Artificial intelligence, Image Processing, and Distributed Computing.



MOHAMMAD ASADPOUR received the B.S. degree in electrical engineering from the University of Tabriz, Iran, in 1993, the M.S. degree in telecommunication engineering from the Ferdowsi University of Mashhad, Mashhad, Iran, in 1996, and the Ph.D. degree in telecommunication engineering from the University of Tabriz, in February 2015.

His research interests include wireless communication, signal processing, channel estimation, power line communication (PLC) channels, and OFDM systems.

• • •